mycloverOS Technical Manuscript

Complete reference for all CloverOS features, services, terminal commands, Service Hub operations, and troubleshooting procedures.

VERSION 0.1.0 (SEEDLING) · MAY 2026 · DEBIAN BOOKWORM · AMD64

Contents

1. System Overview

mycloverOS is a Debian Bookworm-based Linux distribution designed as a self-hosted, modular server operating system. It runs on AMD64 hardware and bundles a full suite of containerized services called **CloverStack**. Every service runs as a Docker container orchestrated through Docker Compose files.

Architecture

CloverOS follows a layered architecture:

- **Base OS:** Debian 12 (Bookworm) with hardened kernel, UFW firewall, fail2ban, and AppArmor
- **Container Runtime:** Docker with overlay2 storage driver, dedicated `cloverstack` bridge network
- **Service Layer (CloverStack):** Modular Docker Compose services in `/opt/cloverstack/modules/`
- **Management Layer:** CloverOS Service Hub (port 7777), Portainer (port 9443), Setup Wizard (port 8080)
- **AI Layer:** Ollama (port 11434) for local LLM inference

Key Directories

PATH	PURPOSE
`/opt/cloverstack/`	CloverStack root — modules, hub, data
`/opt/cloverstack/modules/`	Docker Compose files for each service module
`/opt/cloverstack/hub/`	Service Hub API and dashboard files
`/etc/myclover/`	System configuration (distro.conf, hub.conf, modules.d/)
`/var/lib/cloverstack/`	Persistent state and data
`/var/log/cloverstack/`	Service logs
`/usr/local/bin/`	CloverOS CLI scripts

Default Configuration

SETTING	VALUE
Default user	`clover`
Default hostname	`cloverstack`
Default edition	Server
Default AI model	`phi3:mini` (auto-selects based on RAM)
Default modules	netmon, sentrylog, myclover-vault, chappie, clovermarket
Codename	Seedling (v0.1.0)

2. Installation & First Boot

2.1 Writing the ISO

The CloverOS ISO (`myclover0S-0.1.0-amd64.iso`) is a live Debian image. Write it to a USB drive using one of:

- **Linux/macOS:** `sudo dd if=myclover0S-*.iso of=/dev/sdX bs=4M status=progress`
- **Windows:** Use Rufus or balenaEtcher

Boot from USB. The live system will start with all services available. The setup wizard runs on port 8080.

2.2 Disk Installation

From the live environment, run:

```
sudo myclover-install
```

The installer performs the following steps:

1. Detects target disk (interactive selection)
2. Partitions the disk (GPT+EFI or MBR depending on firmware)
3. Copies the live filesystem via `rsync -aAXH`
4. Creates the user account
5. Removes live-boot packages (`live-boot` , `live-config` , `live-tools`)
6. Switches Docker storage from `vfs` to `overlay2`
7. Switches containerd snapshotter from `native` to `overlayfs`
8. Sets the first-boot provisioning flag
9. Pre-configures the firewall for all CloverStack services
10. Installs GRUB bootloader
11. Ensures openssh-server is installed and enabled

> **Note:** After installation, remove the USB drive and reboot. The first boot will take longer as the provisioning script runs.

2.3 Setup Wizard

The setup wizard is a lightweight Python web server that runs on first boot at `http://<device-ip>:8080`. It provides a web-based interface to:

- Set the hostname
- Configure the admin user and password
- Select which CloverStack modules to enable
- View network interfaces and IP addresses
- Check service status (Docker, SSH, Ollama, UFW, NetworkManager)

The wizard auto-disables after setup completes. The setup-complete flag is stored at `/etc/myclover/.setup-complete`.

Systemd service: `myclover-setup-wizard.service`

2.4 First-Boot Provisioning

The `myclover-provision` script runs automatically on first disk boot. It is triggered by the systemd service `myclover-provision.service`, which checks for the file `/var/lib/cloverstack/.first-boot`.

Provisioning steps:

1. Creates CloverStack directory structure
2. Creates Hub authentication config (`/etc/myclover/hub.conf`) with default password
3. Waits for setup wizard to complete (if running)
4. Starts Docker and creates the `cloverstack` network
5. Pulls the default AI model via Ollama (hardware-aware selection)
6. Installs Portainer (Docker management UI on port 9443)
7. Enables and starts SSH
8. Configures the firewall via `cloverstack-firewall open all`
9. Deploys default modules (netmon, sentrylog, myclover-vault, chappie, clovermarket)
10. Provisions edition-specific components if applicable
11. Removes the first-boot flag

> **Important:** The provisioning script only runs once. If it fails mid-way, you may need to manually complete steps. Check the log at `/var/log/cloverstack/provision.log`.

3. CloverOS Service Hub

The CloverOS Service Hub is the central web-based management dashboard for all CloverStack services. It runs as a Python HTTP server on port 7777.

3.1 Accessing the Hub

Open a browser and navigate to:

```
http://<device-ip>:7777
```

The Hub is served by `clover-hub-api.py` and managed by the systemd service `clover-hub.service`.

Terminal commands:

```
# Check Hub status
sudo systemctl status clover-hub

# Start the Hub
sudo systemctl start clover-hub

# Restart the Hub
sudo systemctl restart clover-hub

# View Hub logs
sudo journalctl -u clover-hub -f
```

3.2 Authentication

The Hub requires login authentication. Sessions use SHA-256 password hashing with 24-hour cookie-based sessions and brute-force rate limiting.

SETTING	VALUE
Default password	`cloverOS`
Config file	`/etc/myclover/hub.conf` (JSON with `password_hash`)
Session duration	24 hours
Rate limiting	Built-in brute-force protection

Changing the password via the Hub: Click the key icon in the dashboard header and enter a new password.

Changing the password via terminal:

```
# Generate new password hash
NEW_HASH=$(echo -n "your-new-password" | sha256sum | awk '{print $1}')

# Update the config
echo "{\"password_hash\": \"${NEW_HASH}\"}" | sudo tee /etc/myclover/hub.conf

# Restart the hub to apply
sudo systemctl restart clover-hub
```

3.3 Managing Services via the Hub

The Hub dashboard displays all CloverStack modules with their status. For each module you can:

- **Start/Stop:** Click the action buttons to start or stop a module's Docker containers
- **Open:** Click the module name to open its web interface in a new tab
- **View Status:** See real-time running/stopped status with port information

Hub API Endpoints:

ENDPOINT	METHOD	DESCRIPTION
`/api/status`	GET	Returns JSON status of all modules
`/api/start/<module>`	POST	Start a module
`/api/stop/<module>`	POST	Stop a module
`/api/login`	POST	Authenticate (JSON body: `{"password":"..."}`)
`/api/logout`	POST	End session
`/api/change-password`	POST	Change password (JSON body with current + new)

4. CloverStack Modules

All CloverStack modules live in `/opt/cloverstack/modules/<module-name>/`. Each module directory contains:

- `docker-compose.yml` — Container definitions and port mappings
- `module.json` — Metadata (name, description, ports, requirements, upstream info)

Universal Module Commands (Terminal)

These commands work for any module:

```
# Start a module
cd /opt/cloverstack/modules/<module>
sudo docker compose up -d

# Stop a module
sudo docker compose down

# Restart a module
sudo docker compose restart

# View logs
sudo docker compose logs -f

# Check status
sudo docker compose ps

# Pull latest images
sudo docker compose pull

# Full restart (recreate containers)
sudo docker compose down && sudo docker compose up -d
```

Using cloverstack-ctl

The `cloverstack-ctl` (aliased to `cloverstack-setup`) provides a unified CLI:

```
# Show status of all modules
sudo cloverstack-ctl status

# Enable a module (marks it to start on boot)
sudo cloverstack-ctl enable <module>

# Disable a module
sudo cloverstack-ctl disable <module>

# Start a specific module
sudo cloverstack-ctl start <module>

# Stop a module
sudo cloverstack-ctl stop <module>

# Restart a module
sudo cloverstack-ctl restart <module>

# View module logs
sudo cloverstack-ctl logs <module>

# List all available modules
sudo cloverstack-ctl list

# Show module info
sudo cloverstack-ctl info <module>
```

4.1 NetMon (Network Monitoring)

NetMon

`Port: 8081 (HTTP) · Upstream: Zabbix 7.0 · Category: Monitoring · Default: Enabled`

Full-featured network monitoring and alerting powered by Zabbix. Monitors hosts, services, bandwidth, and generates alerts. Includes Zabbix Server, Web UI, Agent, and PostgreSQL database.

Access: `http://<device-ip>:8081`

Default credentials: Zabbix default — `Admin` / `zabbix`

Terminal — Start:

```
cd /opt/cloverstack/modules/netmon
sudo docker compose up -d
```

Hub — Start: Click "Start" next to NetMon in the Service Hub dashboard.

Services: zabbix-server, zabbix-web, zabbix-agent, zabbix-postgres

Additional ports: 10051/tcp (Zabbix Server trapper)

Troubleshooting:

- If the web UI shows "Internal Server Error", the PostgreSQL database may not have started yet. Wait 30 seconds and refresh.
- Check container logs: `sudo docker compose logs zabbix-web`
- If Zabbix Agent isn't reporting, verify the container can reach the server: `sudo docker compose exec zabbix-agent zabbix_agentd -t agent.ping`

4.2 SentryLog (Log Management)

SentryLog

`Port: 9000 (HTTP) · Upstream: Graylog 6.0 + Wazuh 4.9 · Category: Security · Default: Enabled`

Centralized log management and security event monitoring. Combines Graylog for log aggregation with Wazuh for threat detection. Includes OpenSearch and MongoDB backends.

Access: `http://<device-ip>:9000`

Terminal — Start:

```
cd /opt/cloverstack/modules/sentrylog
sudo docker compose up -d
```

Hub — Start: Click "Start" next to SentryLog in the dashboard.

Services: graylog, wazuh-manager, graylog-opensearch, graylog-mongo

Additional ports: 1514/tcp (Syslog), 12201/tcp+udp (GELF), 55000/tcp (Wazuh API)

Troubleshooting:

- SentryLog requires at least 2GB RAM. If containers keep restarting, check with `docker stats`.
- OpenSearch needs `vm.max_map_count=262144`. Set with: `sudo sysctl -w vm.max_map_count=262144`
- Persist the setting: `echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf`
- If Graylog fails to start, check MongoDB: `sudo docker compose logs graylog-mongo`

4.3 MyClover Vault (Password Manager)

MyClover Vault

`Port: 8082 (HTTP) · Upstream: Vaultwarden · Category: Security · Default: Enabled`

Self-hosted password manager compatible with Bitwarden clients. Lightweight Rust implementation that supports all Bitwarden features including organizations, attachments, and 2FA.

Access: `http://<device-ip>:8082`

Terminal — Start:

```
cd /opt/cloverstack/modules/myclover-vault
sudo docker compose up -d
```

Hub — Start: Click "Start" next to MyClover Vault.

Services: vaultwarden

Additional ports: 3012/tcp (WebSocket notifications)

Troubleshooting:

- The admin panel is at `/admin`. Admin token is set in the docker-compose environment.

- To use browser extensions, the device IP or a domain name must be used (not `localhost` unless on the device).
- For HTTPS (required for some clients), use Traefik or nginx as a reverse proxy.

4.4 Chappie AI (AI Assistant)

Chappie AI

`Port: 8083 (HTTP) · Upstream: Ollama + Open WebUI · Category: AI · Default: Enabled`

AI assistant with local models via Ollama and Open WebUI. Provides a ChatGPT-like interface for interacting with locally-running language models. No data leaves the device.

Access: `http://<device-ip>:8083`

Terminal — Start:

```
cd /opt/cloverstack/modules/chappie
sudo docker compose up -d
```

Hub — Start: Click "Start" next to Chappie AI.

Enabling additional models:

```
# List available models
ollama list

# Pull a new model
ollama pull llama3.1:8b
ollama pull dolphin-x1

# The model will appear in Open WebUI's model selector automatically
```

GPU requirements: Recommended for inference. Minimum 4GB RAM without GPU, 8GB+ recommended.

Troubleshooting:

- If Open WebUI shows "Ollama not connected", ensure Ollama is running: `systemctl status ollama`
- Restart Ollama: `sudo systemctl restart ollama`
- Check Ollama logs: `journalctl -u ollama -f`
- If models are slow, check available RAM: `free -h`

4.5 CloverMarket (App Marketplace)

CloverMarket

`Port: 8090 (HTTP) · Category: Platform · Default: Enabled`

App marketplace for CloverStack. Browse, install, and manage CloverApps — additional services from the CloverMarket registry.

Access: `http://<device-ip>:8090`

Terminal — CLI usage:

```
# Browse marketplace
sudo clovermarket-ctl browse

# Install an app
sudo clovermarket-ctl install <app-id>

# Check installed apps
sudo clovermarket-ctl status

# Run app startup wizard
sudo clovermarket-ctl wizard <app-id>

# Manage themes
sudo clovermarket-ctl theme list
sudo clovermarket-ctl theme apply <theme-name>
```

Hub — Start: Click "Start" next to CloverMarket.

Registry: `https://market.myclover.tech/api/v1`

4.6 AnythingLLM

AnythingLLM

`Port: 8097 (HTTP) · Upstream: AnythingLLM by Mintplex Labs · Category: AI · Default: Disabled`

AI chat with document workspaces, RAG (Retrieval-Augmented Generation), agents, and multi-user support. Docker-native alternative to Chatbox. Full web UI with document ingestion — drop files in the hotdir volume for automatic ingestion.

Access: `http://<device-ip>:8097`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable anythingllm
cd /opt/cloverstack/modules/anythingllm
sudo docker compose up -d
```

Hub — Start: Click "Start" next to AnythingLLM.

Requirements: 2GB RAM minimum, 10GB disk, GPU recommended.

4.7 Chatbox Team Proxy

Chatbox Team Proxy

`Port: 8095 (HTTP) · Upstream: Chatbox Team Sharing (Caddy proxy) · Category: AI · Default: Disabled`

Shared API proxy for Chatbox desktop clients. Share Ollama or OpenAI access across a team without exposing API keys. Uses Caddy as a reverse proxy.

Access: `http://<device-ip>:8095`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable chatbox-team
cd /opt/cloverstack/modules/chatbox-team
sudo docker compose up -d
```

Use case: Most useful when sharing a cloud API key (OpenAI/Anthropic) across a team. For local Ollama, Chatbox can connect directly.

4.8 CloverBot (AI Support Chatbot)

CloverBot

`Port: 8089 (HTTP) · Upstream: LibreChat · Category: AI · Default: Disabled`

AI-powered customer support chatbot built on LibreChat. Multi-model support with conversation history stored in MongoDB.

Access: `http://<device-ip>:8089`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable cloverbot
cd /opt/cloverstack/modules/cloverbot
sudo docker compose up -d
```

Services: librechat, librechat-mongo

Requirements: 1GB RAM, 10GB disk.

4.9 CloverGuard (DNS Filtering & Web Proxy)

CloverGuard

`Port: 8085 (HTTP) · Upstream: Pi-hole + Squid · Category: Security · Default: Disabled`

DNS-level ad blocking via Pi-hole and web proxy via Squid. Blocks ads, trackers, and malicious domains at the network level. Set your device's DNS to the CloverOS IP to enable network-wide blocking.

Access: `http://<device-ip>:8085/admin` (Pi-hole dashboard)

Terminal — Enable and start:

```
sudo cloverstack-ctl enable cloverguard
cd /opt/cloverstack/modules/cloverguard
sudo docker compose up -d
```

Services: pihole, squid

Ports: 53/tcp+udp (DNS), 8085/tcp (Pi-hole web), 3128/tcp (Squid proxy)

Configuration:

- Point devices' DNS to the CloverOS IP address
- Configure the web proxy at `<device-ip>:3128`
- Manage blocklists via the Pi-hole admin dashboard

Troubleshooting:

- If DNS port 53 is already in use, check for systemd-resolved: `sudo systemctl disable systemd-resolved`
- Test DNS: `dig @<device-ip> google.com`

4.10 CloverSign (Digital Signage)

CloverSign

`Port: 8084 (HTTP) · Upstream: Anthias · Category: Display · Default: Disabled`

Digital signage management system. Schedule and display content (images, videos, web pages) on connected screens.

Access: `http://<device-ip>:8084`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable cloversign
cd /opt/cloverstack/modules/cloversign
sudo docker compose up -d
```

Services: anthias-server, anthias-viewer

4.11 CloverDesign (AI Design Studio)

CloverDesign

`Port: 8088 (HTTP) · Upstream: ComfyUI · Category: AI · Default: Disabled`

AI-powered design studio using ComfyUI. Generate images, run Stable Diffusion workflows, and create visual content locally. Requires a GPU.

Access: `http://<device-ip>:8088`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable cloverdesign
cd /opt/cloverstack/modules/cloverdesign
sudo docker compose up -d
```

Requirements: GPU required, 4GB RAM minimum, 50GB disk for models.

Troubleshooting:

- Verify GPU is accessible: `nvidia-smi` or `ls /dev/dri`
- Check NVIDIA Container Toolkit: `docker run --rm --gpus all nvidia/cuda:12.0-base nvidia-smi`

4.12 CloverPOS (Point of Sale)

CloverPOS

`Port: 8087 (HTTP) · Upstream: ERPNext v15 · Category: Business · Default: Disabled`

Full-featured point of sale and ERP system powered by ERPNext. Inventory management, sales, purchasing, accounting, and HR.

Access: `http://<device-ip>:8087`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable cloverpos
cd /opt/cloverstack/modules/cloverpos
sudo docker compose up -d
```

Services: erpnext, erpnext-worker, erpnext-scheduler, erpnext-mariadb, erpnext-redis-cache, erpnext-redis-queue

Requirements: 4GB RAM, 20GB disk. Heavy service — allow several minutes for first startup.

4.13 CloverMine (Crypto Mining)

CloverMine

`Port: 8086 (HTTP) · Upstream: XMRig · Category: Compute · Default: Disabled (Opt-in)`

Opt-in cryptocurrency mining using XMRig. Includes a web dashboard for monitoring hash rates and earnings.

Access: `http://<device-ip>:8086` (dashboard)

Terminal — Enable and start:

```
sudo cloverstack-ctl enable clovermine
cd /opt/cloverstack/modules/clovermine
sudo docker compose up -d
```

Services: xmrig, clovermine-dashboard

> **Warning:** This module is opt-in only. It will use significant CPU/GPU resources and increase power consumption. Configure mining pool and wallet address before starting.

4.14 CloverMedia (Smart TV & Streaming)

CloverMedia

`Port: 8096 (HTTP) · Upstream: Jellyfin + Navidrome · Category: Media · Default: Disabled`

Smart TV, video streaming, and music server. Jellyfin provides Netflix-like media streaming. Navidrome provides a music streaming server compatible with Subsonic clients.

Access:

- Jellyfin: `http://<device-ip>:8096`
- Navidrome: `http://<device-ip>:4533`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable clovermedia
cd /opt/cloverstack/modules/clovermedia
sudo docker compose up -d
```

Services: jellyfin, navidrome

Additional ports: 4533/tcp (Navidrome), 1900/udp (DLNA discovery)

4.15 StreamServer (Live Streaming)

StreamServer

`Port: 8094 (HTTP) · Upstream: Owncast + MediaMTX · Category: Media · Default: Disabled`

Self-hosted live streaming platform. Owncast provides a Twitch-like streaming experience. MediaMTX handles multi-protocol video relay (RTMP, RTSP, WebRTC, SRT, HLS).

Access: `http://<device-ip>:8094`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable streamserver
cd /opt/cloverstack/modules/streamserver
sudo docker compose up -d
```

Services: owncast, mediamtx

Streaming ports: 1935/tcp (RTMP), 8555/tcp (RTSP), 9710/tcp (SRT)

Streaming with OBS:

```
Stream URL: rtmp://<device-ip>:1935/live
Stream Key: (set in Owncast admin at /admin)
```

4.16 CloverDrone (UAV Control)

CloverDrone

`Port: 8090 (HTTP) · Upstream: MAVProxy + MediaMTX · Category: Robotics · Default: Disabled`

UAV control, mission planning, and video relay. Provides MAVLink-based flight control with real-time video streaming.

Access: `http://<device-ip>:8090`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable cloverdrone
cd /opt/cloverstack/modules/cloverdrone
sudo docker compose up -d
```

Services: mavproxy, cloverdrone-web, cloverdrone-video

Ports: 14550/udp (MAVLink), 8554/tcp (RTSP video)

4.17 CloverMesh (Distributed Networking)

CloverMesh

`Ports: 8091 (API), 8092 (UI) · Upstream: Netmaker · Category: Networking · Default: Disabled`

Distributed WireGuard mesh networking using Netmaker. Connect multiple CloverOS devices into a secure, encrypted mesh network.

Access:

- API: `http://<device-ip>:8091`
- Dashboard: `http://<device-ip>:8092`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable clovermesh
cd /opt/cloverstack/modules/clovermesh
sudo docker compose up -d
```

CLI commands:

```
# Initialize a new mesh cluster
sudo cloverstack-ctl mesh init --name my-cluster --node-ip 192.168.1.10

# Join an existing cluster
sudo cloverstack-ctl mesh join --token <token> --node-ip 192.168.1.20

# Check cluster status
sudo cloverstack-ctl mesh status

# List all nodes
sudo cloverstack-ctl mesh nodes

# Leave the cluster
sudo cloverstack-ctl mesh leave
```

Services: netmaker-server, netmaker-mq, netmaker-ui

Additional port: 51821/udp (WireGuard)

4.18 CloverMesh Radio (LoRa Mesh)

CloverMesh Radio

`Port: 8093 (HTTP) · Upstream: Meshtastic · Category: Communications · Default: Disabled`

LoRa mesh communications via Meshtastic. Enables long-range, low-power radio mesh networking for off-grid communication.

Access: `http://<device-ip>:8093`

Terminal — Enable and start:

```
sudo cloverstack-ctl enable clovermesh-radio
cd /opt/cloverstack/modules/clovermesh-radio
sudo docker compose up -d
```

Services: meshtastic-web, meshtastic-mqtt

Additional port: 1884/tcp (MQTT)

5. Desktop Environments

CloverOS provides containerized desktop environments via the `cloverdesktop` script. Desktops run as Docker containers with VNC/web access, using both LinuxServer webtop images and Kasm images.

Available Desktops

DESKTOP ID	IMAGE	DESCRIPTION	TIER
xfce	linuxserver/webtop:debian-xfce	Lightweight XFCE desktop	Standard
kde	linuxserver/webtop:debian-kde	Full KDE Plasma desktop	Standard
gnome	linuxserver/webtop:debian-gnome	GNOME desktop	Standard
mate	linuxserver/webtop:debian-mate	MATE desktop	Standard
i3	linuxserver/webtop:debian-i3	i3 tiling window manager	Standard
cinnamon	linuxserver/webtop:debian-cinnamon	Cinnamon desktop	Standard
kasm-desktop	kasmweb/desktop:1.15.0	Kasm Full Desktop (GPU-ready)	Premium
kasm-chrome	kasmweb/chrome:1.15.0	Kasm Chrome Browser	Premium
kasm-firefox	kasmweb/firefox:1.15.0	Kasm Firefox Browser	Premium

Commands

```
# Start a desktop
cloverdesktop start <desktop-id>

# Start with custom password
DESKTOP_PASSWORD=mypassword cloverdesktop start kasm-desktop

# List running desktops
cloverdesktop list

# Stop a desktop
cloverdesktop stop <desktop-id>

# Stop all desktops
cloverdesktop stop-all
```

Access

Desktops are accessible via web browser. LinuxServer webtop desktops use HTTPS on the assigned port (starting at 6901). Kasm desktops also use HTTPS on port 6901+.

URL: `https://<device-ip>:6901` (port increments for additional desktops)

Default Credentials

IMAGE TYPE	USERNAME	PASSWORD
Kasm images	`kasm_user`	`changeme` (or value of DESKTOP_PASSWORD)
LinuxServer webtop	N/A (auto-login)	Set via DESKTOP_PASSWORD if provided

Troubleshooting:

- If the desktop won't start, check Docker: `docker ps -a | grep cloverdesktop`
- If the browser shows a certificate warning, this is expected — the containers use self-signed certificates
- For GPU acceleration, ensure `/dev/dri` exists on the host
- Default resolution: 1920x1080. Override with: `DESKTOP_RESOLUTION=2560x1440 cloverdesktop start kde`

6. AI & Local LLM Integration

CloverOS ships with **Ollama** pre-installed as the local LLM inference engine. Ollama runs as a systemd service on port 11434 and automatically selects models based on available hardware.

Ollama Service Management

```
# Check status
systemctl status ollama

# Start/stop/restart
sudo systemctl start ollama
sudo systemctl stop ollama
sudo systemctl restart ollama

# View logs
journalctl -u ollama -f
```

Model Management

```
# List installed models
ollama list

# Pull a model
ollama pull phi3:mini
ollama pull llama3.1:8b
ollama pull "hf.co/dphn/Dolphin-X1-8B-GGUF:Q4_K_M"

# Remove a model
ollama rm phi3:mini

# Run a model interactively
ollama run phi3:mini

# API access
curl http://localhost:11434/api/generate -d '{
  "model": "phi3:mini",
  "prompt": "Hello, world!"
}'
```

Available Models

MODEL	PROVIDER	PARAMETERS	MIN RAM	DESCRIPTION
phi3:mini	Microsoft	3.8B	4GB	Fast, lightweight for constrained hardware
dolphin-x1	dphn (Eric Hartford)	8B	8GB	Uncensored Llama 3.1 fine-tune, no content restrictions
llama3.1:8b	Meta	8B	8GB	Meta's flagship open model, strong general performance

Hardware-Aware Auto-Selection

During provisioning, CloverOS automatically selects the best model based on available RAM:

AVAILABLE RAM	MODEL SELECTED	QUANTIZATION
>10 GB	Dolphin-X1-8B	Q6_K (highest quality)
8-10 GB	Dolphin-X1-8B	Q4_K_M (balanced)
6-8 GB	Dolphin-X1-8B	Q3_K_L (compact)
<6 GB	Phi-3 Mini	Full (3.8B params)

AI Frontends on CloverOS

SERVICE	PORT	USE CASE
Chappie (Open WebUI)	8083	ChatGPT-like web UI, default AI interface
AnythingLLM	8097	Document workspaces, RAG, AI agents
CloverBot (LibreChat)	8089	Customer support chatbot
Chatbox Team Proxy	8095	Shared API access for desktop Chatbox clients

7. Networking & Firewall

CloverOS uses **UFW** (Uncomplicated Firewall) with the `cloverstack-firewall` management script.

Firewall Management

```
# Open all CloverStack ports
sudo cloverstack-firewall open all

# Open ports for a specific module
sudo cloverstack-firewall open netmon
sudo cloverstack-firewall open chappie

# Close ports for a module
sudo cloverstack-firewall close chappie

# Show current CloverStack firewall rules
sudo cloverstack-firewall status

# List all modules and their ports
sudo cloverstack-firewall list
```

Manual UFW Commands

```
# Check UFW status
sudo ufw status verbose

# Allow a specific port
sudo ufw allow 8081/tcp

# Deny a port
sudo ufw deny 8081/tcp

# Delete a rule
sudo ufw delete allow 8081/tcp

# Reset all rules
sudo ufw reset
```

System Ports (Always Open)

PORT	SERVICE
22/tcp	SSH
23/tcp	Telnet
80/tcp	HTTP (nginx)
443/tcp	HTTPS
7777/tcp	CloverOS Service Hub
8080/tcp	Setup Wizard
9443/tcp	Portainer (HTTPS)
11434/tcp	Ollama API
6901-6920/tcp	Desktop environments

Docker Networking

All CloverStack containers run on the `cloverstack` Docker bridge network. Containers can communicate with each other using their service names as hostnames.

```
# Create the network (done automatically during provisioning)
docker network create cloverstack

# List networks
docker network ls

# Inspect the cloverstack network
docker network inspect cloverstack
```

SSH Configuration

```
# Enable SSH
sudo systemctl enable ssh
sudo systemctl start ssh

# Check SSH status
sudo systemctl status ssh

# SSH hardening config location
/etc/ssh/sshd_config.d/90-myclover-hardening.conf
```

SSH hardening features: Strong ciphers only (chacha20-poly1305, aes256-gcm), restricted MAC algorithms, restricted key exchange algorithms, no root login, publickey + password auth.

8. System Scripts Reference

All CloverOS scripts are located at `/usr/local/bin/` and can be run with `sudo`.

SCRIPT	DESCRIPTION
`cloverstack-setup`	CloverStack service orchestrator (enable/disable/start/stop/status modules)
`cloverstack-firewall`	Manage UFW rules for all CloverStack services
`cloverdesktop`	Launch, manage, and export containerized desktop environments
`clovermarket`	CloverMarket app marketplace CLI
`clovermarket-ctl`	CloverMarket management (browse, install, themes)
`clovermesh`	Cluster management — "Fail to Anywhere" mesh networking
`clovernas`	Storage management (ZFS pools, datasets, snapshots, shares)
`clovernas-edition`	NAS edition setup and management
`cloverdeploy`	Container & VM deployment orchestration
`clovervisor`	Hypervisor management (VMs, GPU passthrough, clustering)
`clovercreate`	Creator edition — projects, renders, streaming
`cloverwall`	Firewall edition — nftables rules, VPN, IDS, traffic shaping
`cloverfactory`	Industrial edition — MQTT, Modbus, SCADA, Node-RED
`cloverstrike`	Penetration testing & security audit suite
`clovervr`	VR server management — streaming, headsets, social VR
`cloverapp-picker`	Interactive app selection tool
`myclover-install`	Disk installer (run from live USB)
`myclover-provision`	First-boot provisioning (runs automatically)
`myclover-update`	System updater (packages, containers, scripts)

9. CloverOS Editions

CloverOS ships as a single ISO but supports multiple specialized editions. The edition is determined by installed scripts and configuration files. All editions share the base CloverStack platform.

9.1 Server Edition (Default)

The standard CloverOS installation. Includes all CloverStack modules, AI integration, and the Service Hub. Suitable for home servers, small businesses, and development environments.

Default modules: NetMon, SentryLog, MyClover Vault, Chappie AI, CloverMarket

9.2 CloverVisor (Hypervisor Blackbox Edition)

Enterprise hypervisor with VM management, GPU passthrough, HA clustering, and automated backups. Runs with a read-only root filesystem and no SSH by default ("blackbox" mode).

Trigger: Requires `/usr/local/bin/clovervisor` + `/etc/myclover/clovervisor.d/hypervisor.conf`

Web UI: Port 8006

Features:

- KVM/QEMU virtualization with libvirt
- Open vSwitch networking
- ZFS or LVM-thin storage
- VFIO/IOMMU GPU passthrough
- Corosync/Pacemaker HA clustering
- Restic or ZFS-send backups
- Cockpit web management

CLI:

```
sudo clovervisor vm list
sudo clovervisor vm create --name myvm --ram 4096 --cpus 2 --disk 50G
sudo clovervisor vm start myvm
sudo clovervisor snapshot create myvm
sudo clovervisor gpu list
sudo clovervisor gpu attach myvm 0000:01:00.0
```

SSH in Blackbox mode: SSH is disabled by default. Enable via: `sudo clovervisor ssh enable` or via the web UI under Settings > SSH Access.

9.3 CloverNAS (NAS Edition)

Network-attached storage appliance with ZFS, SMB/NFS shares, snapshot scheduling, and replication.

Trigger: Requires `/usr/local/bin/clovernas-edition` + `/etc/myclover/clovernas.d/nas.conf`

Web UI: Cockpit on port 8080

CLI:

```
# Pool management
sudo cloverstack-ctl storage pool create tank mirror /dev/sda /dev/sdb
sudo cloverstack-ctl storage pool status
sudo cloverstack-ctl storage pool scrub tank

# Dataset management
sudo cloverstack-ctl storage dataset create tank/media
sudo cloverstack-ctl storage dataset list

# Snapshots
sudo cloverstack-ctl storage snap create tank/media daily
sudo cloverstack-ctl storage snap list tank/media
sudo cloverstack-ctl storage snap rollback tank/media@daily

# Shares
sudo cloverstack-ctl storage share create /tank/media --smb --nfs
sudo cloverstack-ctl storage share list
```

9.4 CloverCreate (Creator Edition)

Content creation workstation with project management, rendering, streaming, and mesh render distribution.

Trigger: Requires `/usr/local/bin/clovercreate` + `/etc/myclover/clovercreate.d/creator.conf`

CLI:

```
# Projects
sudo clovercreate project new blender my-animation
sudo clovercreate project list

# Rendering
sudo clovercreate render local scene.blend --frames 1-100
sudo clovercreate render mesh scene.blend --nodes all

# Streaming
sudo clovercreate stream start --key my-stream-key
sudo clovercreate stream status
```

9.5 CloverWall (Firewall Edition)

Enterprise firewall/router with nftables, VPN (WireGuard/OpenVPN), IDS (Suricata), traffic shaping, and HA failover.

Trigger: Requires `/usr/local/bin/cloverwall` + `/etc/myclover/cloverwall.d/firewall.conf`

CLI:

```
# Firewall rules
sudo cloverwall rule list
sudo cloverwall rule add forward -s 192.168.1.0/24 -d 0.0.0.0/0 -j accept

# NAT/port forwarding
sudo cloverwall nat add 443 192.168.1.10:443

# VPN
sudo cloverwall vpn wireguard setup
sudo cloverwall vpn peer add mobile-phone

# IDS
sudo cloverwall ids status
sudo cloverwall ids alerts --tail 50

# Traffic shaping
sudo cloverwall qos set --interface eth0 --upload 100mbit --download 500mbit
```

9.6 CloverFactory (Industrial Edition)

SCADA/ICS/OT management platform with MQTT broker, Modbus gateway, OPC-UA, Node-RED automation, and industrial dashboards.

Trigger: Requires `/usr/local/bin/cloverfactory` + `/etc/myclover/cloverfactory.d/industrial.conf`

CLI:

```
# MQTT
sudo cloverfactory mqtt status
sudo cloverfactory mqtt topics
sudo cloverfactory mqtt subscribe sensors/temperature

# Modbus
sudo cloverfactory modbus scan 192.168.1.100
sudo cloverfactory modbus read 192.168.1.100 0 10

# Node-RED
sudo cloverfactory flows status
sudo cloverfactory flows deploy my-flow.json

# Alarms
sudo cloverfactory alarm list
sudo cloverfactory alarm acknowledge <alarm-id>
```

10. Security Hardening

CloverOS applies multi-layer security hardening during the ISO build process via the `0200-harden.hook.chroot` script.

Hardening Layers

1. **Kernel sysctl hardening:** IP spoofing protection, SYN flood protection, ICMP redirect blocking, source routing disabled, reverse path filtering, TCP timestamps disabled
2. **SSH hardening:** Strong ciphers only, no root login, fail2ban protection
3. **UFW firewall:** Default deny incoming, allow outgoing, specific port allowlist
4. **fail2ban:** SSH brute-force protection with automatic banning
5. **AppArmor:** Mandatory access control profiles for critical services
6. **Audit logging:** auditd rules for monitoring SSH, sudo, and system changes
7. **File permissions:** Restricted access on `/etc/ssh`, sensitive config files

fail2ban Management

```
# Check status
sudo fail2ban-client status

# Check SSH jail
sudo fail2ban-client status sshd

# Unban an IP
sudo fail2ban-client set sshd unbanip <ip-address>

# View banned IPs
sudo fail2ban-client status sshd | grep "Currently banned"
```

Useful Security Aliases

CloverOS includes pre-configured aliases:

```
# View failed login attempts
failed-logins

# This is an alias for:
sudo journalctl _SYSTEMD_UNIT=sshd.service | grep -i "failed\|invalid"
```

11. Updating CloverOS

Full System Update

```
sudo myclover-update
```

This performs a comprehensive update:

1. Updates all CloverOS scripts from GitHub (`jonfulk805-og/mycloverOS`)
2. Updates system packages via `apt-get`
3. Pulls latest Docker images for running containers
4. Updates Ollama models
5. Updates CloverMarket registry

Script-Only Update

```
sudo myclover-update --scripts
```

Updates only the CloverOS CLI scripts from GitHub without touching packages or containers.

Core vs. Optional Scripts

Core scripts (always updated): `cloverstack-setup` , `myclover-update` , `myclover-install` , `myclover-provision`

Optional scripts (updated only if already installed): All other scripts including `clovermarket-ctl` , `clovervisor` , `clovernas` , `cloverdesktop` , etc.

Manual Script Update

```
# Update a specific script manually
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/scripts/
<script-name> \
  -o /usr/local/bin/<script-name>
sudo chmod +x /usr/local/bin/<script-name>
```

Update Hub Files

```
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/overlay/opt/
cloverstack/hub/clover-hub-api.py \
  -o /opt/cloverstack/hub/clover-hub-api.py
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/overlay/opt/
cloverstack/hub/index.html \
  -o /opt/cloverstack/hub/index.html
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/overlay/opt/
cloverstack/hub/login.html \
  -o /opt/cloverstack/hub/login.html
sudo systemctl restart clover-hub
```

Update log: `/var/log/cloverstack/update.log`

12. Troubleshooting

SSH Not Available After Install

Symptom: Port 22 not open after disk install and first boot.

Cause: `apt-get autoremove` during install may remove openssh-server, or provisioning only enabled SSH without starting it.

Fix:

```
sudo apt-get install -y openssh-server
sudo systemctl enable ssh
sudo systemctl start ssh
```

Only Port 80 Shows in Port Scanner

Symptom: Angry IP Scanner or nmap only shows port 80 (nginx).

Cause: Firewall not configured and/or services not started.

Fix:

```
# Install and run the firewall script
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/scripts/
cloverstack-firewall \
  -o /usr/local/bin/cloverstack-firewall
sudo chmod +x /usr/local/bin/cloverstack-firewall
sudo cloverstack-firewall open all

# Start services
cd /opt/cloverstack/modules/netmon && sudo docker compose up -d
cd /opt/cloverstack/modules/sentrylog && sudo docker compose up -d
# ... repeat for other modules
```

Docker Container Won't Start

Symptom: `docker compose up -d` fails or container immediately exits.

Diagnostic steps:

```
# Check container logs
sudo docker compose logs

# Check if image exists
sudo docker images | grep <image-name>

# Pull latest image
sudo docker compose pull

# Check disk space
df -h

# Check memory
free -h

# Check Docker daemon
sudo systemctl status docker
sudo journalctl -u docker -f
```

Service Shows Wrong Content (Port Mismatch)

Symptom: Clicking a service in the Hub shows another service's page.

Cause: Container was started with old docker-compose.yml that had wrong port mappings.

Fix:

```
# Stop and remove old containers
cd /opt/cloverstack/modules/<module>
sudo docker compose down

# Pull updated compose file
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/overlay/opt/
cloverstack/modules/<module>/docker-compose.yml \
  -o docker-compose.yml

# Restart with correct ports
sudo docker compose up -d
```

Hub Not Accessible (Port 7777)

Symptom: Cannot reach `http://<device-ip>:7777`.

Fix:

```
# Check if hub service exists
sudo systemctl status clover-hub

# If not found, create and start it
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/overlay/opt/
cloverstack/hub/clover-hub-api.py \
  -o /opt/cloverstack/hub/clover-hub-api.py
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/overlay/opt/
cloverstack/hub/index.html \
  -o /opt/cloverstack/hub/index.html
sudo curl -fsSL https://raw.githubusercontent.com/jonfulk805-og/mycloverOS/main/overlay/opt/
cloverstack/hub/login.html \
  -o /opt/cloverstack/hub/login.html

# Create the systemd service if missing
sudo tee /etc/systemd/system/clover-hub.service <<'EOF'
[Unit]
Description=CloverOS Service Hub
After=network.target docker.service
Wants=docker.service

[Service]
Type=simple
WorkingDirectory=/opt/cloverstack/hub
ExecStart=/usr/bin/python3 /opt/cloverstack/hub/clover-hub-api.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable clover-hub
sudo systemctl start clover-hub

# Open the port
sudo ufw allow 7777/tcp
```

Ollama Not Responding

Symptom: Chappie/Open WebUI shows "Ollama not connected" or API calls fail.

Fix:

```
# Check Ollama status
systemctl status ollama

# Restart Ollama
sudo systemctl restart ollama

# Test the API
curl http://localhost:11434/api/tags

# If not installed, install it
curl -fsSL https://ollama.ai/install.sh | sh

# Pull a model
ollama pull phi3:mini
```

Desktop Login Failed

Symptom: Kasm desktop shows incorrect password.

Fix: Default Kasm credentials are `kasm_user` / `changeme`. If a custom password was set, use that. To reset:

```
# Stop and restart with a known password
cloverdesktop stop kasm-desktop
DESKTOP_PASSWORD=newpassword cloverdesktop start kasm-desktop
```

Provisioning Failed

Symptom: Services missing after first boot.

Diagnostic:

```
# Check provisioning log
sudo cat /var/log/cloverstack/provision.log

# Check if first-boot flag was consumed
ls -la /var/lib/cloverstack/.first-boot
# If it still exists, provisioning didn't complete

# Re-run provisioning manually
sudo /usr/local/bin/myclover-provision
```

Permission Denied Errors

Most CloverOS commands require root privileges:

```
# Always use sudo for CloverStack commands
sudo cloverstack-ctl status
sudo docker compose up -d
sudo cloverstack-firewall open all
```

Disk Space Full

```
# Check disk usage
df -h

# Check Docker disk usage
docker system df

# Clean up unused Docker resources
docker system prune -a --volumes

# Check largest directories
du -sh /var/lib/docker/*
du -sh /opt/cloverstack/modules/*/
```

Network Not Available

```
# Check NetworkManager
sudo systemctl status NetworkManager
sudo nmcli device status
sudo nmcli connection show

# Check IP address
ip addr show

# Restart networking
sudo systemctl restart NetworkManager
```

13. Complete Port Reference

PORT	PROTOCOL	SERVICE
22	TCP	SSH
23	TCP	Telnet
53	TCP/UDP	DNS (CloverGuard / Pi-hole)
80	TCP	HTTP (nginx)
443	TCP	HTTPS
1514	TCP	Syslog (SentryLog)
1884	TCP	MQTT (CloverMesh Radio)
1900	UDP	DLNA (CloverMedia)
1935	TCP	RTMP (StreamServer)
3012	TCP	WebSocket (MyClover Vault)
3128	TCP	Squid Proxy (CloverGuard)
4533	TCP	Navidrome (CloverMedia)
6901-6920	TCP	Desktop environments (HTTPS)
7777	TCP	CloverOS Service Hub
8080	TCP	Setup Wizard
8081	TCP	NetMon (Zabbix)
8082	TCP	MyClover Vault (Vaultwarden)
8083	TCP	Chappie AI (Open WebUI)
8084	TCP	CloverSign (Anthias)
8085	TCP	CloverGuard (Pi-hole)
8086	TCP	CloverMine dashboard
8087	TCP	CloverPOS (ERPNext)
8088	TCP	CloverDesign (ComfyUI)
8089	TCP	CloverBot (LibreChat)
8090	TCP	CloverMarket / CloverDrone

8091	TCP	CloverMesh API
8092	TCP	CloverMesh UI
8093	TCP	CloverMesh Radio
8094	TCP	StreamServer (Owncast)
8095	TCP	Chatbox Team Proxy
8096	TCP	CloverMedia (Jellyfin)
8097	TCP	AnythingLLM
8554	TCP	RTSP (CloverDrone video)
8555	TCP	RTSP (StreamServer)
9000	TCP	SentryLog (Graylog)
9443	TCP	Portainer (HTTPS)
9710	TCP	SRT (StreamServer)
10051	TCP	Zabbix Server (NetMon)
11434	TCP	Ollama API
12201	TCP/UDP	GELF (SentryLog)
14550	UDP	MAVLink (CloverDrone)
51821	UDP	WireGuard (CloverMesh)
55000	TCP	Wazuh API (SentryLog)

14. Default Credentials

SERVICE	USERNAME	PASSWORD	NOTES
CloverOS (system login)	`clover`	Set during install	Default user, member of sudo and docker groups
Service Hub (port 7777)	N/A	`cloverOS`	Single password, change via Hub UI or terminal
Kasm Desktop	`kasm_user`	`changeme`	Override with `DESKTOP_PASSWORD` env var
Zabbix (NetMon)	`Admin`	`zabbix`	Change after first login
Portainer	Set on first access	Set on first access	Create admin account at first login
Graylog (SentryLog)	`admin`	Set in docker-compose	Check `GRAYLOG_ROOT_PASSWORD_SHA2` env var

> **Security:** Change all default passwords immediately after installation. Use the Service Hub's change password feature and update individual service credentials through their respective web interfaces.